Product Requirements Document

Real-Time Job Intelligence Engine

Version: 2.0 (Free-tier edition) **Stack:** MERN · TypeScript · Bun · TanStack · shadcn/ui · Tailwind CSS · Elasticsearch · Ollama **Status:** Pre-development **Cost to run:** $0/month

---

Table of Contents

1. Project Overview
2. Goals & Success Metrics
3. Free-Tier Infrastructure Strategy
4. Tech Stack Research
5. System Architecture
6. Feature Specifications
7. Data Models
8. API Design
9. Build Phases
10. Folder Structure
11. Environment Variables
12. Open Questions
13. Quick Start Checklist

---

1. Project Overview

A full-stack job search platform combining Elasticsearch hybrid search (BM25 + kNN semantic ranking) with a privacy-first TensorFlow.js resume scorer that runs entirely in the browser. Embeddings are generated locally via Ollama or via Jina AI's generous free tier — zero cost in either case.

One-liner (for your resume/interviews)

> "Built a job search engine with hybrid BM25 + kNN ranking in Elasticsearch and a privacy-first in-browser resume scorer powered by TensorFlow.js — entire stack runs at $0/month using local Ollama embeddings and self-hosted Elasticsearch."

---

2. Goals & Success Metrics

Primary Goals

- Zero ongoing cost — every service either runs locally or has a permanent free tier
- Demonstrate mastery of Elasticsearch index design, hybrid queries, and Percolate
- Show in-browser ML with TensorFlow.js (rare in portfolios, very impressive)
- Cover CI/CD, webhooks, TanStack full ecosystem, TypeScript end-to-end
- Live, publicly accessible URL

Success Metrics

Metric	Target
Monthly infrastructure cost	$0
Job index size	500+ real jobs at launch
Search latency (p95)	< 200ms
TF.js scoring (50 results)	< 150ms in browser
CI pipeline	< 3 min on push
Webhook delivery	< 10s after new match

---

3. Free-Tier Infrastructure Strategy

Every piece of this stack is free. No trials, no expiry, no credit card needed for the core services.

3.1 Elasticsearch — Local Docker + Railway self-hosted

There is no free-forever managed Elasticsearch equivalent to Upstash. The solution is to self-host it, which is straightforward.

Local dev — Docker Compose:

```yaml
```

```
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.13.0
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
  ports: ["9200:9200"]
  volumes: ["esdata:/usr/share/elasticsearch/data"]
```

Needs ~1GB RAM in Docker. Set Docker Desktop → Settings → Resources → Memory to at least 4GB total.

Production — Railway self-hosted Docker:

- Deploy the same image to Railway using its Docker deployment
- Set `ES_JAVA_OPTS=-Xms512m -Xmx512m` and `discovery.type=single-node` as Railway env vars
- Add a Railway volume to persist the index across restarts
- Railway gives $5/month free credit; ES at 512MB heap costs ~$3-4/month → effectively free forever

---

3.2 Embeddings — Ollama (local) + Jina AI free tier (production)

This replaces OpenAI entirely. Zero cost.

Option A: Ollama + nomic-embed-text (local dev)

Ollama is a free open-source tool that runs embedding models locally as a REST server on `localhost:11434`. The `nomic-embed-text` model outperforms OpenAI `text-embedding-ada-002` and `text-embedding-3-small` on both short and long context tasks. It produces **768-dimensional** vectors.

```bash
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh   # Linux/macOS
# Windows: download installer from https://ollama.com

# Pull the model (~274MB download, one-time)
ollama pull nomic-embed-text

# Test it
curl http://localhost:11434/api/embed \
  -d '{"model": "nomic-embed-text", "input": "senior react engineer remote"}'
```

Option B: Jina AI free tier (recommended for both local and prod)

Jina AI gives every new API key **10 million free tokens** at signup — no credit card required. At ~200 tokens per job description, that covers 50,000 jobs before spending a cent. This is the easier path because it works the same way in both dev and prod, avoiding an embedding dimension mismatch.

- **Signup:** https://jina.ai (no credit card)
- **Model:** `jina-embeddings-v3` → **1024-dimensional** vectors
- **Rate limit (free):** 100 RPM, 100K TPM
- **Dimensions:** 1024

Recommended decision: Use Jina AI everywhere

Using Jina AI in both dev and prod keeps your ES index mapping consistent (always 1024 dims). Document Ollama as an offline fallback. The `embedder.ts` module handles both:

```typescript
// server/src/lib/embedder.ts
export async function embed(texts: string[]): Promise<number[][]> {
  if (process.env.EMBEDDING_PROVIDER === 'ollama') {
    const res = await fetch(`${process.env.OLLAMA_URL}/api/embed`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ model: 'nomic-embed-text', input: texts }),
    })
    const { embeddings } = await res.json()
    return embeddings
  }

  // Default: Jina AI
  const res = await fetch('https://api.jina.ai/v1/embeddings', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.JINA_API_KEY}`,
    },
    body: JSON.stringify({ model: 'jina-embeddings-v3', input: texts }),
  })
  const { data } = await res.json()
  return data.map((d: any) => d.embedding)
}
```

Set `EMBEDDING_PROVIDER=ollama` in `.env` if you want fully offline dev. Leave it unset (default Jina) otherwise.

3.3 MongoDB — Atlas M0 (free forever)

- Signup: https://cloud.mongodb.com → "Try Free" → M0 tier
- 512MB storage, no credit card, no expiry
- Locally: Docker Compose `mongo:7` container
- Prod: Atlas M0 connection string

3.4 Redis — Upstash free tier

This is where Upstash actually shines for this project.

- Signup: https://upstash.com → create Redis database → free tier
- 10,000 commands/day, 256MB — more than enough for the BullMQ webhook queue
- Locally: Docker Compose `redis:7-alpine` container
- Prod: Upstash `rediss://` connection string

3.5 Full cost summary

Service	Local	Production	Monthly cost
Elasticsearch	Docker	Railway self-hosted	$0
Embeddings	Jina AI free / Ollama	Jina AI free (10M tokens)	$0
MongoDB	Docker	Atlas M0	$0
Redis	Docker	Upstash free	$0
API server	localhost	Railway (~$1-2/mo from $5 credit)	$0
Frontend	localhost	Vercel	$0
Total			**$0/month**

---

4. Tech Stack Research

Runtime & Tooling

Bun

- Replaces Node + npm + esbuild + Jest in one binary
- Built-in TypeScript execution: `bun run src/index.ts` with no compilation step
- 3-4x faster package installs
- Gotcha: some native Node addons may not work. Use `bun --bun` flag or fall back to `node` for specific scripts
- Railway supports Bun natively

```bash
bun init
bun add <pkg>
bun run src/index.ts    # runs TypeScript directly
bun test
bun build ./src/index.ts --outdir ./dist --target bun
```

TypeScript

- `"strict": true` everywhere
- Separate `tsconfig.json` for client and server
- Use `zod` for runtime validation that produces TypeScript types automatically

---

Frontend

React 19 + Vite

- Pure SPA — no Server Components complexity
- `@/` path alias in `vite.config.ts`

TanStack Router

- Type-safe file-based routing
- Every filter lives in the URL as typed, validated search params

```typescript
```

```typescript
// routes/search.tsx
export const Route = createFileRoute('/search')({
  validateSearch: z.object({
    q: z.string().default(''),
    location: z.string().optional(),
    salary_min: z.number().optional(),
    sort: z.enum(['relevance', 'date', 'salary']).default('relevance'),
  }),
})
```

TanStack Query v5

```typescript
useQuery({ queryKey: ['jobs', searchParams], queryFn: () => searchJobs(searchParams)
useMutation({ mutationFn: saveSearch, onSuccess: () => queryClient.invalidateQueries
```

TanStack Table + TanStack Virtual

- Table handles data model; Virtual renders only visible rows
- ~10 DOM nodes regardless of 500 results

```typescript
const virtualizer = useVirtualizer({
  count: rows.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => 88,
})
```

shadcn/ui + Tailwind CSS v4

- shadcn CLI copies Radix UI components into your codebase — you own them
- `bunx --bun shadcn@latest add button card input dialog badge command sheet slider`
- Tailwind v4: CSS-based config, no `tailwind.config.js`
- Key components for this project: `Command` (search), `Sheet` (detail panel), `Badge` (match %), `Slider` (salary filter)

Backend

Express.js (on Bun)

- Router → Controller → Service pattern
- Middleware: `cors`, `helmet`, `express-rate-limit`, zod validation

MongoDB + Mongoose

- Stores: users, saved searches, webhook subscriptions, delivery log
- Does NOT store jobs — those live in Elasticsearch
- `InferSchemaType` for automatic TypeScript types

Elasticsearch Client

```typescript
import { Client } from '@elastic/elasticsearch'
export const esClient = new Client({ node: process.env.ES_URL })
// No auth needed locally (security disabled)
// Railway: keep security disabled, protected by Railway's private networking
```

ML / AI Layer

TensorFlow.js (Browser — resume scorer)

- Model: Universal Sentence Encoder (USE) — pre-trained, 512-dim, no training needed
- Runs in a Web Worker to avoid blocking the UI thread
- Important: USE runs independently of the server-side Jina embeddings. The browser re-embeds both the resume AND the job description text (returned in the API response) using USE, then computes cosine similarity locally

```typescript
```

```
// workers/resume.worker.ts
import * as use from '@tensorflow-models/universal-sentence-encoder'

let model: use.UniversalSentenceEncoder | null = null

self.onmessage = async ({ data }) => {
  if (!model) model = await use.load()   // ~400ms cold start, then cached

  const { resumeChunks, jobDescriptions } = data
  const allTexts = [...resumeChunks, ...jobDescriptions]
  const embeddings = await model.embed(allTexts)
  const embArray = await embeddings.array()

  const resumeEmbs = embArray.slice(0, resumeChunks.length)
  const jobEmbs = embArray.slice(resumeChunks.length)

  const scores = jobEmbs.map(jobEmb =>
    Math.max(...resumeEmbs.map(resumeEmb => cosineSim(resumeEmb, jobEmb)))
  )
  self.postMessage({ scores })
}

function cosineSim(a: number[], b: number[]): number {
  const dot = a.reduce((sum, ai, i) => sum + ai * b[i], 0)
  const magA = Math.sqrt(a.reduce((s, x) => s + x * x, 0))
  const magB = Math.sqrt(b.reduce((s, x) => s + x * x, 0))
  return dot / (magA * magB)
}
```

PDF Parsing

- `pdfjs-dist` (Mozilla PDF.js) — runs entirely client-side
- Extract text → split into 3 chunks (summary, skills section, recent role) → pass to Web Worker

---

Infrastructure & DevOps

Docker Compose (full local stack)

```yaml
```

```yaml
version: '3.8'
services:
  api:
    build: ./server
    ports: ["3001:3001"]
    env_file: server/.env
    depends_on: [mongodb, elasticsearch, redis]

  mongodb:
    image: mongo:7
    ports: ["27017:27017"]
    volumes: ["mongodata:/data/db"]

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.13.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
    ports: ["9200:9200"]
    volumes: ["esdata:/usr/share/elasticsearch/data"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

volumes:
  mongodata:
  esdata:
```

Note: Ollama runs on your host machine (not Docker). Use `OLLAMA_URL=http://host.docker.internal:11434` when using Docker Compose.

GitHub Actions CI/CD

```yaml
```

```yaml
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v2
      - run: bun install
      - run: bun run typecheck
      - run: bun run lint
      - run: bun run test
      - run: bun run build
      - name: Deploy to Railway
        if: github.ref == 'refs/heads/main'
        run: railway up
        env:
          RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}
```

---

5. System Architecture

```
┌──────────────────────────────────────────────────────┐
│              CLIENT (Vite + React)                   │
│  TanStack Router · Query · Table · Virtual           │
│  shadcn/ui · Tailwind v4                             │
│  TF.js USE model (Web Worker, 512-dim)               │
│  pdfjs-dist (PDF parsing, client-side)               │
└──────────────────┬───────────────────────────────────┘
                   │ REST / JSON
┌──────────────────▼───────────────────────────────────┐
│            API SERVER (Bun + Express)                │
│  /api/v1/jobs  /api/v1/search                        │
│  /api/v1/saved-searches  /api/v1/webhooks            │
└──────┬──────────────────┬──────────────────┬─────────┘
       │                  │                  │
┌──────▼──────┐  ┌────────▼────────┐  ┌──────▼──────────────┐
│  MongoDB    │  │Elasticsearch    │  │  embedder.ts        │
│  Atlas M0   │  │  Docker /       │  │  → Jina AI (free)   │
│  (free)     │  │  Railway        │  │  → Ollama (local)   │
└──────┬──────┘  └─────────────────┘  └─────────────────────┘
       │
┌──────▼──────────────────┐      ┌──────────────────────────┐
│  Redis (Upstash         │──────▶│  Webhook Fanout          │
```

```
│  free tier)              │      │  BullMQ + HMAC-SHA256   │
│  BullMQ queue            │      │  → Slack / custom URL   │
└──────────────────────────┘      └─────────────────────────┘
```

Data Flow: Search Request

1. User types query → URL search params update (TanStack Router)
2. TanStack Query fires `GET /api/v1/jobs/search?q=...`
3. Express embeds query via `embedder.ts` (Jina AI or Ollama)
4. Hybrid BM25 + kNN query fires against ES → top 50 results returned including `description` text
5. Client renders results; if resume uploaded → Web Worker embeds resume + descriptions via USE → cosine similarity → re-ranks list
6. TanStack Virtual renders visible rows only

Data Flow: Job Alert

1. `node-cron` triggers ingestion every 6 hours
2. Fetch sources → normalize → embed → bulk index into ES
3. Percolate query finds which saved searches match each new job
4. Matched jobs enqueued in BullMQ (Upstash Redis)
5. BullMQ worker fires HMAC-signed POST to each subscriber URL
6. Delivery outcome logged in MongoDB

---

6. Feature Specifications

Feature 1: Job Search

Elasticsearch hybrid query (RRF):

```json
```

```json
{
  "retriever": {
    "rrf": {
      "retrievers": [
        {
          "standard": {
            "query": {
              "multi_match": {
                "query": "{{q}}",
                "fields": ["title^3", "description", "company", "skills"]
              }
            }
          }
        },
        {
          "knn": {
            "field": "embedding",
            "query_vector": "{{queryEmbedding}}",
            "num_candidates": 100,
            "k": 50
          }
        }
      ],
      "rank_window_size": 50
    }
  }
}
```

Components: `SearchBar` (shadcn `Command`), `JobList` (TanStack Virtual), `JobCard`, `FilterPanel`

URL params (TanStack Router): `q`, `location`, `salary_min`, `salary_max`, `remote`, `sort`, `page`

Feature 2: Resume Scoring (In-Browser, Privacy-First)

Job descriptions are included as plain text in the API response. Both resume and job descriptions are embedded via TF.js USE in the browser. Nothing leaves the client.

Components: `ResumeUploader` (react-dropzone), `MatchBadge` (green >70%, amber >40%, gray below), `SkillGapSidebar`, `ResumeWorker` (Web Worker)

Scoring: max cosine similarity across 3 resume chunks vs. each job's description embedding — all computed locally via USE 512-dim

Privacy note for README:

> "Resume scoring runs entirely in your browser using TensorFlow.js. Your resume text is never sent to any server."

Feature 3: Saved Searches + Webhooks (Percolate)

Register a saved search as a Percolate query:

```typescript
await esClient.index({
  index: 'jobs-percolator',
  id: savedSearch._id.toString(),
  document: {
    query: {
      multi_match: { query: savedSearch.query, fields: ['title', 'description', 'ski
    }
  }
})
```

Find matching saved searches for a new job:

```typescript
const matches = await esClient.search({
  index: 'jobs-percolator',
  body: { query: { percolate: { field: 'query', document: newJob } } }
})
```

Webhook payload:

```typescript
interface JobAlertWebhook {
  event: 'job.match'
  version: '1'
  saved_search_id: string
  timestamp: string   // ISO 8601
  job: { id: string; title: string; company: string; url: string; posted_at: string
}
// Header: X-Webhook-Signature: sha256=<hmac-sha256>
```

HMAC signing:

```typescript
```

```
import crypto from 'crypto'
const sig = 'sha256=' + crypto.createHmac('sha256', secret).update(body).digest('hex
```

Feature 4: Job Detail Panel

shadcn `Sheet` slide-over. Preserves search state (no navigation). Contents: full description (react-markdown), skill match breakdown if resume uploaded, Apply link, bookmark, share URL.

Feature 5: Ingestion Pipeline

Free job data sources:

- `GET https://remotive.com/api/remote-jobs` — free JSON API (~300 remote jobs)
- HN Who's Hiring via `https://hn.algolia.com/api/v1/search` — monthly thread
- `GET https://www.arbeitnow.com/api/job-board-api` — free EU/remote jobs

Pipeline:

1. Fetch → normalize to `JobDocument`
2. Deduplicate: `source_url` term query in ES
3. Embed descriptions via `embedder.ts`
4. `esClient.bulk()` index
5. Percolate → push matches to BullMQ
6. `node-cron` triggers every 6 hours

Feature 6: Auth

JWT (email + password), `httpOnly` cookie storage, `bcryptjs` for password hashing.

---

7. Data Models

MongoDB Schemas

```typescript
```

```
interface User {
  _id: ObjectId; email: string; passwordHash: string; createdAt: Date
}

interface SavedSearch {
  _id: ObjectId; userId: ObjectId; name: string; query: string
  filters: { location?: string[]; salary_min?: number; remote?: boolean }
  percolatorId: string; createdAt: Date; lastAlertAt?: Date
}

interface WebhookSubscription {
  _id: ObjectId; userId: ObjectId; savedSearchId: ObjectId
  url: string; secret: string; active: boolean; createdAt: Date
}

interface WebhookDelivery {
  _id: ObjectId; subscriptionId: ObjectId; jobId: string; payload: object
  responseStatus?: number; error?: string; sentAt: Date; success: boolean
}
```

Elasticsearch Index Mapping

```json
{
  "mappings": {
    "properties": {
      "id":          { "type": "keyword" },
      "title":       { "type": "text", "analyzer": "english",
                      "fields": { "keyword": { "type": "keyword" } } },
      "description": { "type": "text", "analyzer": "english" },
      "company":     { "type": "keyword" },
      "location":    { "type": "keyword" },
      "remote":      { "type": "boolean" },
      "salary_min":  { "type": "integer" },
      "salary_max":  { "type": "integer" },
      "skills":      { "type": "text", "analyzer": "english" },
      "source_url":  { "type": "keyword" },
      "posted_at":   { "type": "date" },
      "embedding":   { "type": "dense_vector", "dims": 1024, "index": true, "similar
    }
  },
  "settings": { "number_of_shards": 1, "number_of_replicas": 0 }
}
```

`dims: 1024` = Jina AI `jina-embeddings-v3`. If you switch to Ollama `nomic-embed-text`, change to `dims: 768` and re-index. Decide before Phase 1.

---

8. API Design

All routes prefixed `/api/v1`.

Method	Route	Auth	Description
GET	`/jobs/search`	No	Hybrid search with filters
GET	`/jobs/:id`	No	Single job detail
POST	`/auth/register`	No	Create account
POST	`/auth/login`	No	Get JWT cookie
GET	`/saved-searches`	Yes	List saved searches
POST	`/saved-searches`	Yes	Save + register Percolate
DELETE	`/saved-searches/:id`	Yes	Remove saved search
GET	`/webhooks`	Yes	List subscriptions
POST	`/webhooks`	Yes	Register endpoint
DELETE	`/webhooks/:id`	Yes	Remove webhook
POST	`/webhooks/:id/test`	Yes	Fire test delivery
GET	`/webhooks/:id/deliveries`	Yes	Delivery log
POST	`/admin/ingest`	Admin	Trigger manual ingestion

---

9. Build Phases

Phase 1 — Local ES + Ingestion (Week 1–2)

Goal: 500+ real jobs in local Elasticsearch. No cloud required yet.

- [ ] `docker-compose up` — ES, MongoDB, Redis running
- [ ] Sign up for Jina AI free key (https://jina.ai) — no credit card
- [ ] Create `jobs` index with 1024-dim mapping
- [ ] Create `jobs-percolator` index

- [ ] Write `scripts/ingest.ts` — fetch Remotive → embed via Jina → bulk index
- [ ] Verify hybrid query works: `curl "http://localhost:9200/jobs/_search" -d '...'`

Exit criteria: 500+ jobs in index. Hybrid BM25 + kNN query returns sensible results.

Phase 2 — Backend API (Week 2–3)

Goal: REST API on ES. MongoDB auth.

- [ ] Scaffold Bun + Express + TypeScript server
- [ ] `GET /jobs/search` with hybrid ES RRF query
- [ ] `GET /jobs/:id`
- [ ] JWT register/login
- [ ] Saved searches endpoints + Percolate registration
- [ ] Zod request validation middleware

Exit criteria: Search, auth, saved searches all work via Postman/curl.

Phase 3 — Frontend UI (Week 3–5)

Goal: Working search UI with all filters and job detail.

- [ ] Scaffold Vite + React + TanStack Router
- [ ] shadcn/ui + Tailwind v4 setup
- [ ] `SearchBar` (shadcn `Command`)
- [ ] `JobList` with TanStack Virtual
- [ ] TanStack Query for all data fetching
- [ ] URL-synced filters via `validateSearch`
- [ ] `JobCard` + `FilterPanel`
- [ ] Job detail `Sheet` panel

Exit criteria: Search works in browser. URL is shareable with exact filter state.

Phase 4 — TF.js Resume Scorer (Week 5–6)

Goal: Upload resume → match % on every result.

- [ ] `pdfjs-dist` for client-side PDF text extraction
- [ ] `ResumeWorker` Web Worker with USE model
- [ ] Handle USE cold-start (~400ms) with loading state
- [ ] `ResumeUploader` drag-drop component
- [ ] `MatchBadge` on `JobCard`

- [ ] `SkillGapSidebar` in detail panel

Exit criteria: Upload a real PDF → every result shows a match %.

Phase 5 — Webhooks + BullMQ (Week 6–7)

Goal: Save a search → Slack notification on new match.

- [ ] BullMQ webhook queue (Redis-backed)
- [ ] HMAC signing utility
- [ ] Webhook CRUD endpoints
- [ ] Percolate integration in ingestion pipeline
- [ ] Retry logic in BullMQ worker (exponential backoff)
- [ ] `WebhookSettings` frontend page with delivery log
- [ ] Test with real Slack Incoming Webhook URL

Exit criteria: New job indexed → Slack message in < 10 seconds.

Phase 6 — Deploy + CI/CD + Polish (Week 7–8)

Goal: Live URL. Automated pipeline. Interview-ready README.

- [ ] GitHub Actions: typecheck → lint → test → build → Railway deploy
- [ ] Frontend on Vercel
- [ ] Backend + ES on Railway (Jina API key as env var)
- [ ] MongoDB Atlas M0
- [ ] Upstash Redis
- [ ] CI badge in README
- [ ] Architecture diagram in README
- [ ] "Technical Decisions" section (explain Jina vs Ollama, BM25+kNN, Percolate)
- [ ] Loom demo video (2 min)

Exit criteria: `git push main` → CI green → live URL auto-updated.

---

10. Folder Structure

```
job-intelligence-engine/
├── client/
│   ├── src/
│   │   ├── routes/
│   │   │   ├── __root.tsx
│   │   │   ├── index.tsx
```

```
│   │   │   ├── search.tsx
│   │   │   └── settings/webhooks.tsx
│   │   ├── components/
│   │   │   ├── ui/                  # shadcn (auto-generated, do not edit)
│   │   │   ├── SearchBar.tsx
│   │   │   ├── JobList.tsx
│   │   │   ├── JobCard.tsx
│   │   │   ├── FilterPanel.tsx
│   │   │   ├── ResumeUploader.tsx
│   │   │   ├── MatchBadge.tsx
│   │   │   └── SkillGapSidebar.tsx
│   │   ├── workers/
│   │   │   └── resume.worker.ts     # TF.js USE + cosine similarity
│   │   ├── hooks/
│   │   │   ├── useJobSearch.ts
│   │   │   └── useResumeScoring.ts
│   │   └── lib/api.ts
│   ├── vite.config.ts
│   └── tsconfig.json
│
├── server/
│   ├── src/
│   │   ├── routes/
│   │   ├── controllers/
│   │   ├── services/
│   │   │   ├── jobService.ts        # ES hybrid query logic
│   │   │   ├── ingestService.ts     # Fetch + embed + bulk index
│   │   │   ├── percolateService.ts  # Save/match percolate queries
│   │   │   └── webhookService.ts    # HMAC + BullMQ dispatch
│   │   ├── models/                  # Mongoose schemas
│   │   ├── middleware/
│   │   │   ├── auth.ts
│   │   │   └── validate.ts
│   │   ├── lib/
│   │   │   ├── elasticsearch.ts     # Client singleton
│   │   │   ├── mongodb.ts
│   │   │   ├── redis.ts             # BullMQ connection
│   │   │   └── embedder.ts          # Jina AI / Ollama unified interface
│   │   └── index.ts
│   ├── scripts/
│   │   └── ingest.ts                # Standalone run: bun run scripts/ingest.ts
│   └── tsconfig.json
│
├── docker-compose.yml
├── .github/workflows/ci.yml
└── README.md
```

11. Environment Variables

```bash
```

```
# server/.env  (local dev)

# MongoDB
MONGODB_URI=mongodb://localhost:27017/jobengine

# Elasticsearch
ES_URL=http://localhost:9200
ES_INDEX=jobs
ES_PERCOLATOR_INDEX=jobs-percolator

# Embeddings — choose one provider
EMBEDDING_PROVIDER=jina              # "jina" or "ollama"
JINA_API_KEY=jina_xxxxxxxxxxxx        # free at jina.ai, no credit card
OLLAMA_URL=http://localhost:11434     # only needed if EMBEDDING_PROVIDER=ollama
EMBEDDING_DIMS=1024                   # 1024 for jina, 768 for ollama nomic-embed-te

# JWT
JWT_SECRET=at_least_32_random_characters_here
JWT_EXPIRES_IN=7d

# Redis
REDIS_URL=redis://localhost:6379

# Server
PORT=3001
NODE_ENV=development

# --- server/.env.production (Railway) ---

MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/jobengine
ES_URL=http://elasticsearch.railway.internal:9200
EMBEDDING_PROVIDER=jina
JINA_API_KEY=jina_xxxxxxxxxxxx
EMBEDDING_DIMS=1024
REDIS_URL=rediss://default:xxxx@xxxx.upstash.io:6379
NODE_ENV=production

# client/.env
VITE_API_URL=http://localhost:3001/api/v1

# client/.env.production
VITE_API_URL=https://your-api.railway.app/api/v1
```

12. Open Questions

#	Question	Decide by
1	Jina AI (1024-dim) everywhere, or Ollama (768-dim) locally + Jina in prod?	Before Phase 1 — affects index mapping
2	Add Google OAuth alongside email/password?	Phase 3
3	Add streaming "AI summarize this job" using OpenRouter free models?	Phase 5
4	Build a public "trending skills this week" page as a bonus feature?	Phase 6
5	Build a browser extension that scores any job page against your resume?	Post-launch

---

13. Quick Start Checklist

Run this before writing any app code:

Create free accounts (15 min total):

- [ ] Jina AI — https://jina.ai → get API key (10M free tokens, no card)
- [ ] MongoDB Atlas — https://cloud.mongodb.com → create M0 cluster (free forever)
- [ ] Upstash — https://upstash.com → create Redis database (free tier)
- [ ] Railway — https://railway.app → connect GitHub ($5/month free credit)
- [ ] Vercel — https://vercel.com → connect GitHub (free)

Local setup:

```bash
```

```bash
# 1. Start all local services
docker-compose up -d

# 2. Verify Elasticsearch is running
curl http://localhost:9200

# 3. Create the jobs index
curl -X PUT http://localhost:9200/jobs \
  -H 'Content-Type: application/json' \
  -d '{
    "mappings": {
      "properties": {
        "embedding": { "type": "dense_vector", "dims": 1024, "index": true, "similar
        "title": { "type": "text", "analyzer": "english" },
        "description": { "type": "text", "analyzer": "english" },
        "company": { "type": "keyword" },
        "location": { "type": "keyword" },
        "remote": { "type": "boolean" },
        "salary_min": { "type": "integer" },
        "source_url": { "type": "keyword" },
        "posted_at": { "type": "date" }
      }
    },
    "settings": { "number_of_shards": 1, "number_of_replicas": 0 }
  }'

# 4. Run the ingestion script (after writing it in Phase 1)
cd server && bun run scripts/ingest.ts

# 5. Check job count
curl http://localhost:9200/jobs/_count
# Should return {"count": 500+, ...}
```

PRD v2.0 — Free-tier edition — Last updated March 2026